

Core Java Interview Question with Answer

Q.1. You wrote a Java program but it's running slower than expected. How would you start debugging the issue?

Answer:

If my Java program is running slower than expected, I would approach the problem step by step in a structured way.

First, I would check the **logic and time complexity** of the code, especially loops or nested loops, because inefficient algorithms are one of the most common reasons for performance issues.

Next, I would analyze the **data structures being used**. For example, if I am using a list for frequent searching operations, I might replace it with a HashMap or HashSet to improve performance.

Then, I would check for **unnecessary object creation**, because creating too many objects increases memory usage and puts pressure on the garbage collector, which can slow down the application.

After that, I would use **logging or profiling tools** to identify which part of the code is taking the most time.

In real-world applications, performance issues usually come from:

- Incorrect data structures
- Inefficient algorithms
- High memory usage

So I mainly focus on optimizing these areas step by step.

Q.2. What happens internally when you run a Java program?

Answer:

When we run a Java program, the process happens in multiple steps.

First, the .java source file is compiled by the Java compiler into **bytecode**, which is stored in a .class file.

This bytecode is then executed by the **JVM, which stands for Java Virtual Machine**.

Inside the JVM, a component called the **ClassLoader** loads the required classes into memory. After that, the **execution engine** executes the bytecode line by line.

One important point is that Java is **platform-independent**, because the same bytecode can run on any system that has a JVM installed.

So the overall flow is:

Source code → Compilation → Bytecode → JVM → Execution

Q.3. You are designing a system for different types of vehicles. How would you structure your classes?

Answer:

If I have to design a system for different types of vehicles, I would use object-oriented programming principles.

I would create a base class called Vehicle that contains common properties like speed and methods like start() or stop().

Then I would create child classes like Car and Bike that inherit from the Vehicle class and can override methods if needed.

This approach uses:

- **Inheritance** for code reuse
- **Polymorphism** to allow different behaviors for the same method

This design makes the system flexible and scalable. For example, if I want to add a new vehicle type in the future, I can do it without changing existing code.

Q.4. In a real project, when would you prefer composition over inheritance?

Answer:

I prefer composition over inheritance when there is a **“has-a” relationship instead of an “is-a” relationship**.

For example, a car has an engine, but a car is not an engine.

So instead of extending a class, I would create a separate Engine class and include it inside the Car class as an object.

This approach is better because:

- It reduces tight coupling
 - It makes the code more flexible
 - Changes in one class do not affect other classes In real-world applications, composition is often preferred for better maintainability.
-

Q.5. How would you explain polymorphism using a real-world example?

Answer:

Polymorphism means using a single interface or method name with different implementations.

A simple real-world example is a payment system.

We can have a method called `pay()`, but its behavior changes depending on whether the payment is done using UPI, credit card, or wallet.

So even though the method name is the same, the implementation is different.

This helps in writing clean and scalable code, because we don't need to create different method names for each type of operation.

Q.6. You need fast lookup and no duplicates. Which collection would you use and why?

Answer:

If I need fast lookup and want to avoid duplicates, I would use a **HashSet** or **HashMap**.

These data structures are based on hashing, which provides **$O(1)$ average time complexity** for search operations.

For example:

- HashSet is used when we only need to store unique values
- HashMap is used when we need key-value pairs

This is very useful in real-world applications like caching, storing unique records, or fast data retrieval.

Q.7. Difference between ArrayList and LinkedList in real usage?

Answer:

The choice between ArrayList and LinkedList depends on the use case.

- **ArrayList** is better when we need fast access to elements using an index, because it is based on a dynamic array.
- **LinkedList** is better when there are frequent insertions and deletions, because it does not require shifting elements.

In real-world scenarios:

- ArrayList is used more commonly
 - LinkedList is used in specific cases like implementing queues or frequent modifications
-

Q.8. Why is HashMap fast? What happens internally?

Answer:

HashMap is fast because it uses **hashing**.

When we insert a key:

- A hash function converts it into an index
- Data is stored in a bucket

This allows direct access instead of searching linearly.

Even in case of collisions, Java handles it using:

- Linked list or tree structure internally

That's why average time complexity is $O(1)$.

Q.9. What will happen if you don't override hashCode() and equals()?

Answer:

If we do not override equals() and hashCode(), Java compares objects based on their memory reference instead of their actual data.

This can cause problems in collections like HashSet and HashMap, such as:

- Duplicate objects being stored
- Incorrect behavior during search operations

So it is important to override both methods to ensure:

- Objects are compared based on their actual values
- Hash-based collections work correctly

Q.10. Difference between checked and unchecked exceptions with real example?

Answer:

Exceptions in Java are divided into checked and unchecked exceptions.

- **Checked exceptions** are checked at compile time, which means the compiler forces us to handle them. For example, file handling exceptions like `IOException`.
- **Unchecked exceptions** occur at runtime, such as `NullPointerException` or `ArithmeticException`.

In real-world applications:

- Checked exceptions are used for situations where recovery is possible
- Unchecked exceptions usually indicate programming errors

So both are important, but they are used in different scenarios.
